Lecture plan · for the lecturer

Assessing programming at scale

Programming · University · Lecture 4

Time	Materials
60 min	Projector or board; The lecture slides and the seminar sheet from the kit; Phones or laptops with Darsi, for the lab

Learning goals

- Design strong test suites.
- Give useful feedback.
- Protect integrity by design.

Activity: Break the submission

1. Warm-up (5 min): Show the buggy median and ask students whether the visible test catches it.
2. Test (20 min): Pairs write hidden tests that break three passing submissions.
3. Feedback (15 min): Write a hint for each failing case that does not give the fix.
4. Policy (15 min): Draft a one-paragraph policy and one live-modification task.
5. Wrap-up (5 min): Students write the muddiest point of the lecture and one question; read them before next week.

Discussion

- Which test broke the most submissions?
- When does a hint become a fix?
- What will your policy allow?

Exit ticket

- Write one hidden test for your next assignment.

Try it in your next lecture
Show a submission that passes the visible tests and ask pairs to write one hidden test that breaks it; keep the best tests for next term's suite.

Seminar sheet · for students

Assessing programming at scale

Programming · University · Lecture 4

Name _____ Class _____ Date _____

1 **Which change best shows a student understands their submitted code?**

◯ An AI-detection tool ◯ More visible tests ◯ Modify it live and explain

2 **Fix the median.**

For s = sorted(xs) of even length n, the median is the mean of s[n // 2 - 1] and s[_____].

For [1, 2, 3, 4] it is _____ .

Word bank: n // 2 · n · 3 · 2.5

3 **Which strengthen an autograded assignment? Choose all that apply.**

☐ Hidden edge-case tests ☐ Showing the full expected output of every hidden test

☐ Checking broken reference solutions fail ☐ A short code-reading or oral component

4 **A high similarity score from MOSS is sufficient proof of misconduct.**

◯ True ◯ False

5 **Match each problem to a design response.**

Code hard-codes visible answers	• • • •	Mutation testing
Endless trial-and-error resubmits	• • • •	Live modification and explanation
AI-written code not understood	• • • •	Submission limits
Weak test suite	• • • •	Hidden tests

✎ **What is your current policy on AI assistance in programming assignments, and is it written where students see it?**